ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO BÀI TẬP THỰC HÀNH 06

GIAO TIẾP PHI NGÔN NGỮ VÀ ĐIỀU KHIỂN TURTLEBOT 4 BẰNG CỬ CHỈ TAY AI

NON-VERBAL HUMAN–ROBOT INTERACTION USING HAND GESTURE RECOGNITION (MEDIAPIPE & OPENCV)

Sinh viên thực hiện: Nguyễn Quang Tuân – 23020767

Giảng viên hướng dẫn: PGS.TS. Hoàng Văn Xiêm
CN. Nguyễn Quốc Bảo

Hà Nội – 2026

Mục lục

1 TỔNG QUAN HỆ THỐNG & SƠ ĐỒ KIẾN TRÚC **1**
1.1 Mục tiêu bài thực hành 1
1.2 Cấu trúc thư mục gói chương trình trong `src/` 1
1.3 Sơ đồ kiến trúc hệ thống (System Architecture) 2

2 GIẢI THÍCH CÁC ROS 2 NODE, TOPIC VÀ LUỒNG XỬ LÝ **3**
2.1 Vai trò của các ROS 2 Node trong hệ thống 3
2.2 Bảng phân tích chi tiết các Topic truyền thông ROS 2 4
2.3 Sơ đồ Máy trạng thái hữu hạn (FSM) và luồng xử lý điều khiển 4

3 BẢNG ÁNH XẠ CỬ CHỈ TAY VÀ HÀNH VI ROBOT **4**
3.1 Bảng ánh xạ 06 cử chỉ tay và thông số điều khiển động học 4
3.2 Cơ chế chống nhiễu, dừng khẩn cấp và đảm bảo an toàn 5
3.3 Trích đoạn mã nguồn cốt lõi 5

4 KẾT QUẢ THỰC NGHIỆM & HÌNH ẢNH MINH HỌA **7**
4.1 Hình ảnh minh họa kết quả thực nghiệm toàn hệ thống 7
4.2 Log Terminal thực nghiệm từ quá trình vận hành 7

5 HƯỚNG DẪN THỰC THI & ĐƯỜNG DẪN NỘP BÀI **8**
5.1 Hướng dẫn biên dịch và khởi chạy chương trình 8
5.2 Liên kết Video Demo và GitHub Repository 8

1 TỔNG QUAN HỆ THỐNG & SƠ ĐỒ KIẾN TRÚC

1.1 Mục tiêu bài thực hành

Bài thực hành số 6 xây dựng hệ thống tương tác Người – Robot phi ngôn ngữ (*Non-verbal Human–Robot Interaction*) cho phép nhận diện cử chỉ bàn tay theo thời gian thực qua webcam laptop bằng trí tuệ nhân tạo (**MediaPipe Gesture Recognizer** kết hợp **OpenCV**) và điều khiển hành vi chuyển động của robot tự hành **TurtleBot 4 Standard** trong môi trường mô phỏng **Gazebo Fortress** cùng bản đồ định vị **RViz2** thông qua nền tảng **ROS 2 Humble**:

1. **Phần 1 – Khởi chạy Simulation và Webcam:** Khởi chạy TurtleBot 4 trong môi trường Gazebo (`warehouse`), nạp bản đồ 2D đã lưu lên RViz2, tự động rời trạm sạc ra vùng rộng thoáng, kiểm tra robot nhận lệnh vận tốc qua topic `/cmd_vel` và sử dụng OpenCV để đọc/hiển thị hình ảnh thời gian thực từ webcam laptop.

2. **Phần 2 – Nhận diện cử chỉ tay bằng AI (`gesture_recognition_node`):** Đọc khung hình từ webcam bằng OpenCV, sử dụng mô hình **MediaPipe Gesture Recognizer** (`gesture_recognizer.task`) kết hợp phân tích động học 21 điểm khớp bàn tay (*21 Hand Landmarks*) để phát hiện và phân loại chính xác **06 cử chỉ tay**, hiển thị tên cử chỉ cùng độ tin cậy nhận diện trên cửa sổ webcam và publish lên topic `/detected_gesture` (`std_msgs/msg/String`).

3. **Phần 3 – Điều khiển hành vi TurtleBot 4 (`robot_behavior_node`):** Subscribe kết quả nhận diện từ `/detected_gesture` và điều khiển TurtleBot 4 qua topic `/cmd_vel` bằng máy trạng thái hữu hạn (*Finite State Machine – FSM*) tuân thủ nghiêm ngặt cơ chế **chọn hành vi trước – xác nhận bằng `Thumbs_Up` sau**, chống nhiễu kích hoạt lặp lệnh, tự động dừng ngay khi nhận cử chỉ dừng (`Open_Palm`) hoặc khi mất tín hiệu điều khiển (*Watchdog Timeout*), và giám sát khoảng cách an toàn qua LiDAR (`/scan`).

1.2 Cấu trúc thư mục gói chương trình trong `src/`

Toàn bộ mã nguồn của bài tập được đóng gói trong package `turtlebot4_practice4`:

```
turtlebot4_ws/src/
|-- turtlebot4_practice4/
|   |-- package.xml                          # Khai bao phu thuoc ROS 2 (rclpy, std_msgs, geometry_msgs...)
|   |-- setup.py                             # Cau hinh entry_points cho cac ROS 2 Nodes
|   |-- turtlebot4_practice4/
|   |   |-- webcam_cmdvel_check.py           # (Phan 1) Kiem tra /cmd_vel va hien thi Webcam OpenCV
|   |   |-- gesture_recognition_node.py      # (Phan 2) Node AI nhan dien 06 cu chi tay (MediaPipe)
|   |   |-- robot_behavior_node.py           # (Phan 3) Node FSM dieu khien hanh vi TurtleBot 4
|   |   |-- auto_undock.py                   # Tu dong Undock va dua robot ra khu vuc trung tam rong rai
|   |   `-- tf_relay.py                      # Dong bo TF va cung cap mock hardware services
|   |-- launch/
|   |   |-- sim_gesture_control.launch.py    # Launch tich hop: Gazebo + Map/AMCL + RViz2 + Gesture AI
|   |   `-- sim_robot.launch.py              # Launch Gazebo Simulation co ban
|   |-- config/
|   |   |-- gesture_recognizer.task          # Model AI MediaPipe Gesture Recognizer (Float16 TFLite)
|   |   |-- localization_params.yaml         # Cau hinh tham so AMCL va Map Server
|   |   |-- fastdds_udp_only.xml             # Cau hinh FastDDS UDP-only chong loi SHM deadlock
|   |   `-- turtlebot4_nav2.rviz             # Giao dien RViz2 hien thi Map, RobotModel, Scan & Camera
|   |-- maps/
|   |   `-- map.yaml & map.pgm               # Ban do Occupancy Grid 2D da luu tu cac bai truoc
|   |-- scripts/
```

```
|   |   |-- run_gesture_part1_sim_webcam.sh   # Script chay Phan 1: Kiem tra /cmd_vel & Webcam OpenCV
|   |   |-- run_gesture_part2_recognition.sh  # Script chay Phan 2: Node gesture_recognition_node
|   |   |-- run_gesture_part3_behavior.sh     # Script chay Phan 3: Node robot_behavior_node
|   |   `-- run_gesture_all.sh                # Script chay tich hop toan bo he thong trong 1 lenh
|   `-- docs/
|       `-- report.tex & bao_cao_thuc_hanh_04.(pdf|md|html)
`-- README.md                                 # Tai lieu huong dan cai dat, build va chay chuong trinh
```

1.3 Sơ đồ kiến trúc hệ thống (System Architecture)

Hình 1 biểu diễn kiến trúc phân tầng của hệ thống giao tiếp phi ngôn ngữ và điều khiển TurtleBot 4 bằng cử chỉ tay AI:



Hình 1: Sơ đồ kiến trúc tổng thể hệ thống nhận diện cử chỉ tay AI và điều khiển hành vi TurtleBot 4

2 GIẢI THÍCH CÁC ROS 2 NODE, TOPIC VÀ LUỒNG XỬ LÝ

2.1 Vai trò của các ROS 2 Node trong hệ thống

1. **Node `webcam_cmdvel_check` (`webcam_cmdvel_check.py` – Phần 1):**
   - Kiểm tra kết nối các topic của TurtleBot 4 trong Gazebo (`/odom`, `/scan`, `/cmd_vel`).
   - Tự động gọi dịch vụ `/motion_control/set_parameters` đặt `safety_override = 'full'` trên đế Create 3 để mở khóa giới hạn lùi mặc định, cho phép robot nhận cả vận tốc tiến ($v > 0$) và lùi ($v < 0$).
   - Phát chuỗi lệnh vận tốc kiểm thử (tiến, lùi, xoay trái–phải, dừng) lên `/cmd_vel` và mở cửa sổ OpenCV hiển thị hình ảnh trực tiếp từ webcam laptop (`/dev/video0`).
2. **Node `gesture_recognition_node` (`gesture_recognition_node.py` – Phần 2):**
   - Đọc luồng video từ webcam laptop ở độ phân giải 640×480 bằng `cv2.VideoCapture(0)` và lật gương ngang (`cv2.flip`) để thuận tiện tương tác trực diện.
   - Đưa ảnh RGB qua mô hình **MediaPipe Gesture Recognizer** (`gesture_recognizer.task`) để trích xuất 21 điểm khớp bàn tay (*21 Hand Landmarks*) và phân loại cử chỉ tĩnh, đồng thời kết hợp bộ phát hiện dao động ngang cổ tay (*Dynamic Wave Detector*) để nhận diện động tác **Vẫy tay** (`Wave`).
   - Áp dụng bộ lọc chống nhiễu theo thời gian (*Temporal Debounce Filter*) yêu cầu cử chỉ xuất hiện ổn định và vượt ngưỡng độ tin cậy $\geq 58\%$ trước khi chốt kết quả.
   - Hiển thị trực quan khung xương bàn tay, hộp giới hạn, tên cử chỉ, độ tin cậy (%), thanh khóa chống nhiễu (*Anti-Noise Lock*) và trạng thái hiện tại của robot lên cửa sổ OpenCV; đồng thời publish tên cử chỉ lên topic `/detected_gesture` (`std_msgs/msg/String`).
3. **Node `robot_behavior_node` (`robot_behavior_node.py` – Phần 3):**
   - Subscribe topic `/detected_gesture` (`std_msgs/msg/String`), `/scan` (`LaserScan`) và `/odom` (`Odometry`).
   - Vận hành máy trạng thái hữu hạn (**FSM**) 4 trạng thái (`IDLE`, `SELECTED`, `EXECUTING`, `STOPPED`) điều khiển vận tốc robot qua topic `/cmd_vel` ở tần số 20 Hz.
   - Đảm bảo robot **chỉ thực thi hành vi sau khi nhận được cử chỉ xác nhận** `Thumbs_Up`, dừng tức khắc khi nhận `Open_Palm` hoặc khi mất tín hiệu quá 1.8 s, và hủy hành vi về trạng thái chờ khi nhận `Closed_Fist`.
4. **Node `auto_undock` cùng `map_server` & `amcl`:**
   - Nạp bản đồ `maps/map.yaml` đã lưu từ các tuần trước lên RViz2, định vị chính xác robot trên bản đồ, sau đó tự động thực hiện `/undock` và tiến thêm 0.90 m ra vùng trung tâm rộng thoáng để robot thực hiện mọi quỹ đạo mà không va chạm với trạm sạc hay tường.

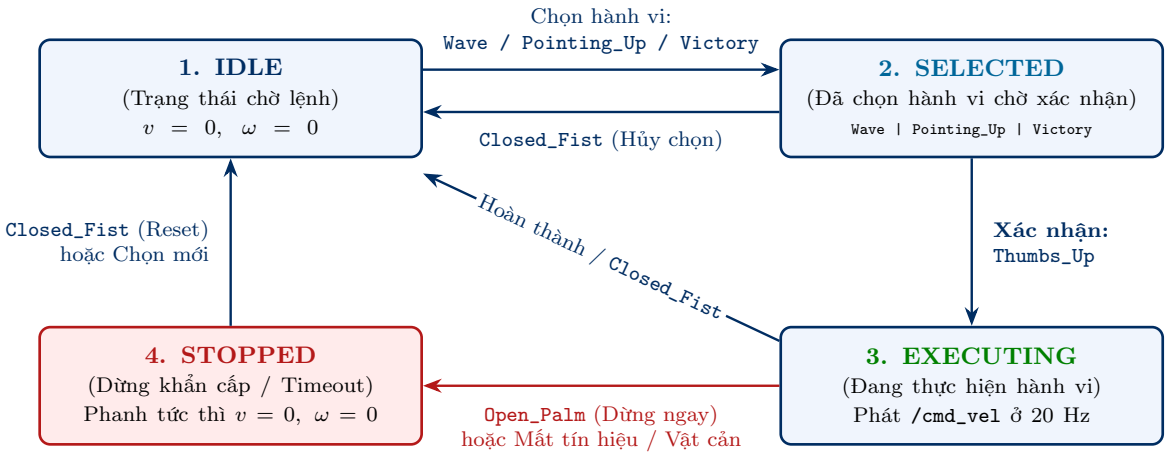
2.2 Bảng phân tích chi tiết các Topic truyền thông ROS 2

Tên Topic	Kiểu Message	Tần số	Luồng dữ liệu và vai trò trong hệ thống
`/detected_gesture`	`std_msgs/msg/String`	15 Hz	**Topic chính (Phần 2 → Phần 3):** Truyền nhãn cử chỉ tay đã lọc chống nhiễu (`Wave`, `Pointing_Up`, `Victory`, `Thumbs_Up`, `Open_Palm`, `Closed_Fist`, `None`).
`/cmd_vel`	`geometry_msgs/msg/Twist`	20 Hz	**Topic điều khiển vận tốc:** Truyền vận tốc dài v_x (`linear.x`) và vận tốc góc ω_z (`angular.z`) từ `robot_behavior_node` xuống bộ điều khiển vi sai của TurtleBot 4.
`/robot_behavior_status`	`std_msgs/msg/String`	Khi đổi / 2.5 Hz	Truyền trạng thái FSM hiện tại của robot ngược về `gesture_recognition_node` để hiển thị trực tiếp lên cửa sổ Webcam OpenCV.
`/scan`	`sensor_msgs/msg/LaserScan`	10 Hz	Dữ liệu quét LiDAR 360° dùng cho AMCL định vị trên bản đồ RViz2 và giám sát chống va chạm (dist < 0.25 m) trong `robot_behavior_node`.
`/odom`	`nav_msgs/msg/Odometry`	20–62 Hz	Phản hồi vị trí (x, y) và góc hướng yaw thực tế của robot trong quá trình di chuyển.
`/map`	`nav_msgs/msg/OccupancyGrid`	Transient	Bản đồ lưới chiếm dụng 2D (`map.yaml` & `map.pgm`) hiển thị trực quan trên giao diện RViz2.

Bảng 1: Danh sách các ROS 2 Topic cốt lõi và luồng trao đổi dữ liệu trong hệ thống

2.3 Sơ đồ Máy trạng thái hữu hạn (FSM) và luồng xử lý điều khiển

Để đảm bảo yêu cầu *"Robot chỉ thực hiện hành vi sau khi nhận được cử chỉ xác nhận"*, luồng xử lý trong `robot_behavior_node` được thiết kế theo mô hình Máy trạng thái hữu hạn (Hình 2):



Hình 2: Sơ đồ chuyển trạng thái FSM trong `robot_behavior_node`

3 BẢNG ÁNH XẠ CỬ CHỈ TAY VÀ HÀNH VI ROBOT

3.1 Bảng ánh xạ 06 cử chỉ tay và thông số điều khiển động học

Hệ thống nhận diện và xử lý đầy đủ **06 cử chỉ tay** (vượt yêu cầu tối thiểu 05 cử chỉ của đề bài), được tổng hợp chi tiết trong Bảng 2:

Cử chỉ tay	**Nhãn trên Topic**	**Phương pháp nhận diện AI (MediaPipe + Landmarks)**	**Hành vi của TurtleBot 4 & Thông số `/cmd_vel`**
1. Vẫy tay (*Wave*)	`Wave`	Phát hiện bàn tay mở (≥ 3 ngón duỗi) có dao động lắc ngang cổ tay ($\Delta x \geq 0.055$, quãng đường ngang ≥ 0.115 trong 0.85 s) hoặc cử chỉ `ILoveYou`.	**Chọn hành vi: Robot xoay trái–phải để chào lại.** Khi xác nhận: thực hiện 4 pha xoay đối xứng ($\omega = \pm 0.85$ rad/s) giúp robot lắc chào 2 chu kỳ và trở về đúng góc ban đầu.
2. Giơ 1 ngón tay (*Pointing Up*)	`Pointing_Up`	Mô hình MediaPipe phân loại `Pointing_Up` (kết hợp kiểm tra hình học chỉ duy nhất ngón trỏ số 8 duỗi thẳng).	**Chọn hành vi: Tiến–lùi 4 lần.** Khi xác nhận: lặp 4 chu kỳ liên tiếp, mỗi chu kỳ tiến thẳng ($v = +0.20$ m/s trong 1.8 s), nghỉ 0.35 s, lùi về ($v = -0.20$ m/s trong 1.8 s), nghỉ 0.35 s.
3. Giơ 2 ngón tay (*Victory*)	`Victory`	Mô hình MediaPipe phân loại `Victory` (2 ngón trỏ và giữa duỗi chữ V, áp út và út gập).	**Chọn hành vi: Di chuyển theo quỹ đạo tròn.** Khi xác nhận: phát đồng thời $v = 0.18$ m/s và $\omega = 0.45$ rad/s tạo quỹ đạo tròn bán kính $R = \frac{v}{\omega} = 0.40$ m khép kín 360° ($T \approx 14.0$ s).
4. Giơ ngón cái (*Thumbs Up*)	`Thumbs_Up`	Mô hình MediaPipe phân loại `Thumb_Up` với độ tin cậy $\geq 58\%$.	**Xác nhận và thực hiện hành vi đã chọn:** Chuyển FSM từ `SELECTED` sang `EXECUTING` để bắt đầu chạy hành vi đã chọn.
5. Xòe bàn tay (*Open Palm*)	`Open_Palm`	Mô hình MediaPipe phân loại `Open_Palm` khi bàn tay mở giữ tĩnh (không lắc ngang).	**Robot dừng ngay lập tức (*Emergency Stop*):** Lập tức phát $v = 0.0, \omega = 0.0$ lên `/cmd_vel`, ngắt chuyển động và chuyển sang `STOPPED`.
6. Nắm tay (*Closed Fist*)	`Closed_Fist`	Mô hình MediaPipe phân loại `Closed_Fist` (cả 5 ngón tay nắm kín).	**Hủy hành vi hiện tại và trở về trạng thái chờ:** Xóa hành vi đang chạy hoặc đang chọn, dừng robot ($v = 0, \omega = 0$) và đưa FSM về `IDLE`.

Bảng 2: Bảng ánh xạ chi tiết 06 cử chỉ tay sang hành vi điều khiển TurtleBot 4

3.2 Cơ chế chống nhiễu, dừng khẩn cấp và đảm bảo an toàn

1. **Chống nhiễu hai tầng (*Two-stage Anti-Noise & Edge-Trigger Cooldown*):**
   - Tại `gesture_recognition_node`: Bộ đệm trượt `gesture_history` lọc đa số qua `debounce_frames = 4` khung hình liên tiếp kết hợp ngưỡng độ tin cậy `min_confidence = 0.58`, loại bỏ hoàn toàn hiện tượng nhấp nháy nhãn khi người dùng đang chuyển đổi tư thế ngón tay.
   - Tại `robot_behavior_node`: Áp dụng cơ chế *Edge-Trigger* kết hợp thời gian khóa lặp `command_cooldown = 1.2s` và `min_confirm_frames = 2`, đảm bảo việc giữ nguyên tay trước webcam không gây kích hoạt lệnh lặp đi lặp lại.
2. **Tự động dừng khi mất tín hiệu điều khiển (*Watchdog Timer*):** Trong vòng lặp điều khiển 20 Hz, nếu robot đang ở trạng thái `EXECUTING` mà thời gian kể từ bản tin `/detected_gesture` cuối cùng vượt quá `signal_timeout = 1.8s` (ví dụ node webcam bị đóng hoặc mất kết nối camera), hệ thống lập tức phanh robot ($v = 0, \omega = 0$) và chuyển sang trạng thái `STOPPED`.
3. **Di chuyển trong khu vực rộng thoáng và chống va chạm bằng LiDAR:** Khi khởi chạy, node `auto_undock` tự động đưa robot lùi khỏi dock và tiến thêm 0.90 m ra chính giữa khoảng trống rộng của nhà kho `warehouse`. Ngoài ra, `robot_behavior_node` liên tục kiểm tra tia quét `/scan` ở góc $\pm 30°$ phía trước (khi tiến) và phía sau (khi lùi), tự động dừng nếu phát hiện vật cản ở cự ly < 0.25 m.

3.3 Trích đoạn mã nguồn cốt lõi

Đoạn mã 1: Trích đoạn phát hiện cử chỉ Vẫy tay động (Wave) và phân loại 06 cử chỉ trong `gesture_recognition_node.py`

```
def _detect_dynamic_wave(self, landmarks, now: float) -> tuple:
    """Phat hien cu chi Vay tay (Wave) dua tren dao dong ngang cua ban tay mo."""
    palm_x = 0.5 * (landmarks[0].x + landmarks[9].x)
    self.hand_x_history.append((now, palm_x))
    while self.hand_x_history and (now - self.hand_x_history[0][0]) > self.wave_window_sec:
        self.hand_x_history.popleft()
    if len(self.hand_x_history) < 6:
        return False, 0.0

    xs = [pt[1] for pt in self.hand_x_history]
    span_x = max(xs) - min(xs)
    path_x = sum(abs(xs[i] - xs[i - 1]) for i in range(1, len(xs)))
    if span_x >= self.wave_min_span_x and path_x >= self.wave_min_path_x:
        self.last_wave_detected_time = now
        conf = min(0.99, 0.82 + (span_x - self.wave_min_span_x) * 1.5)
        return True, conf
    return False, 0.0
```

Đoạn mã 2: Trích đoạn xử lý xác nhận `Thumbs_Up` và tính toán vận tốc trong `robot_behavior_node.py`

```
def _handle_confirm_behavior(self, now: float):
    """Chi thuc thi hanh vi sau khi nhan duoc cu chi xac nhan Thumbs_Up."""
    if self.state == self.STATE_EXECUTING or self.selected_behavior is None:
        return
    self.active_behavior = self.selected_behavior
    self.state = self.STATE_EXECUTING
    self.exec_start_time = now
    self.get_logger().info(f'[XAC NHAN THANH CONG - Thumbs_Up] Bat dau:
        {self.active_behavior}')
```

4 KẾT QUẢ THỰC NGHIỆM & HÌNH ẢNH MINH HỌA

4.1 Hình ảnh minh họa kết quả thực nghiệm toàn hệ thống

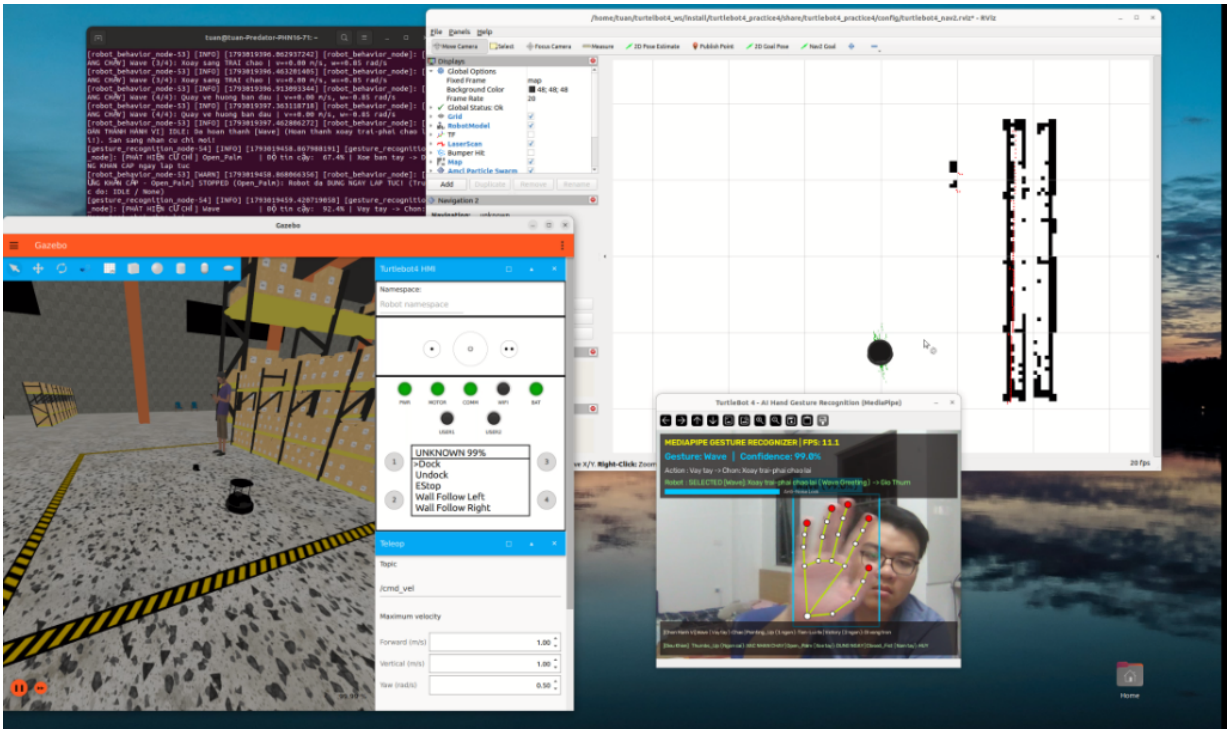
Hình 3 ghi lại toàn cảnh hệ thống đang vận hành thực tế đồng thời trên cả 04 cửa sổ mà hoàn toàn không sử dụng bàn phím:

1. **Góc dưới bên phải (Cửa sổ Webcam AI – MediaPipe Gesture Recognizer):** Hiển thị khung xương 21 điểm khớp bàn tay của người điều khiển với kết quả nhận diện `Gesture: Wave | Confidence: 99.0%`, thanh khóa chống nhiễu `Anti-Noise Lock` đạt 100% và dòng trạng thái đồng bộ trực tiếp từ robot: `Robot: SELECTED [Wave]: Xoay trai-phai chao lai -> Gio Thumbs_Up de XAC NHAN!`.

2. **Góc dưới bên trái (Môi trường mô phỏng Gazebo Fortress):** Robot TurtleBot 4 Standard đã tự động rời trạm sạc (Undock) và tiến ra khu vực sàn rộng rãi có vạch kẻ an toàn, không vướng bất kỳ vật cản nào khi thực hiện xoay chào, tiến–lùi 4 lần và đi quỹ đạo tròn.

3. **Góc trên bên phải (Giao diện RViz2 tích hợp bản đồ `map.yaml`):** Hiển thị bản đồ 2D đã lưu từ các tuần trước, mô hình 3D `RobotModel`, tia quét LiDAR `/scan` khớp với tường bản đồ và đám mây hạt định vị `Amcl Particle Swarm` màu xanh lá hội tụ quanh robot.

4. **Góc trên bên trái (Terminal Log thời gian thực):** Ghi lại nhật ký hoạt động của `gesture_recognition_node` và `robot_behavior_node` khi hoàn thành hành vi `Wave` (4/4 pha xoay chào trái–phải), kích hoạt dừng khẩn cấp `Open_Palm` và chọn lại hành vi mới.



Hình 3: Kết quả thực nghiệm tích hợp: Terminal Log (trái trên), Gazebo Simulation với TurtleBot 4 ở khu vực rộng thoáng (trái dưới), RViz2 hiển thị bản đồ đã lưu & AMCL (phải trên), và cửa sổ Webcam nhận diện cử chỉ tay MediaPipe đạt độ tin cậy 99.0% (phải dưới)

4.2 Log Terminal thực nghiệm từ quá trình vận hành

Đoạn mã 3: Trích xuất Log Terminal thực tế khi nhận diện cử chỉ và điều khiển hành vi TurtleBot 4

```
[robot_behavior_node] [INFO]: [DANG CHAY] Wave (3/4): Xoay sang TRAI chao | v=+0.00 m/s, w=+0.85 rad/s
[robot_behavior_node] [INFO]: [DANG CHAY] Wave (4/4): Quay ve huong ban dau | v=+0.00 m/s, w=-0.85 rad/s
[robot_behavior_node] [INFO]: [HOAN THANH HANH VI] IDLE: Da hoan thanh [Wave] (Hoan thanh xoay trai-phai chao lai!). San sang nhan cu chi moi!
[gesture_recognition_node] [INFO]: [PHAT HIEN CU CHI] Open_Palm    | Do tin cay:  67.4% | Xoe ban tay -> DUNG KHAN CAP ngay lap tuc
[robot_behavior_node] [WARN]: [DUNG KHAN CAP - Open_Palm] STOPPED (Open_Palm): Robot da DUNG NGAY LAP TUC! (Truoc do: IDLE / None)
[gesture_recognition_node] [INFO]: [PHAT HIEN CU CHI] Wave         | Do tin cay:  92.4% | Vay tay -> Chon: Xoay trai-phai chao lai
[robot_behavior_node] [INFO]: [DA CHON HANH VI] Wave (Xoay trai-phai chao lai (Wave Greeting)) | Robot dang cho cu chi Gio ngon cai (Thumbs_Up) de xac nhan thuc thi.
```

5 HƯỚNG DẪN THỰC THI & ĐƯỜNG DẪN NỘP BÀI

5.1 Hướng dẫn biên dịch và khởi chạy chương trình

Đoạn mã 4: Các lệnh biên dịch và khởi chạy hệ thống

```
# 1. Build workspace
cd /home/tuan/turtelbot4_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install --packages-select turtlebot4_practice4
source install/setup.bash

# 2. Chay tich hop toan bo (Gazebo + Map/AMCL tren RViz2 + Auto Undock + 2 Nodes AI):
./src/turtlebot4_practice4/scripts/run_gesture_all.sh

# 3. Hoac chay rieng tung phan:
# Phan 1 - Kiem tra /cmd_vel va hien thi Webcam bang OpenCV:
./src/turtlebot4_practice4/scripts/run_gesture_part1_sim_webcam.sh

# Phan 2 - Khoi chay Node nhan dien cu chi tay AI (gesture_recognition_node):
./src/turtlebot4_practice4/scripts/run_gesture_part2_recognition.sh

# Phan 3 - Khoi chay Node dieu khien hanh vi TurtleBot 4 (robot_behavior_node):
./src/turtlebot4_practice4/scripts/run_gesture_part3_behavior.sh
```

5.2 Liên kết Video Demo và GitHub Repository

- **GitHub Repository (Public):**
  https://github.com/Tuan404cl/Turtlebot_sim_test3.git
- **Video Demo trên Google Drive (Chế độ công khai):**
  https://drive.google.com/drive/folders/1vl4OOC9A4C3Ed_HwHmJAeVCzwEQZulCx?usp=drive_link